UCD School of Electrical and Electronic Engineering

EEEN40280
Digital and Embedded Systems

System on Chip Design

Introduction

In this assignment you will build a complete embedded system on an FPGA, to work with other hardware on a Digilent Nexys-4/A7 board. The system will use an ARM Cortex-M0 processor, connected to various peripheral blocks using an AHB-Lite bus, running software written mostly in C. Most of the peripheral blocks are provided, but you will have to integrate those blocks into the system, making the necessary connections to the bus and other signals.

Schedule

You have five lab sessions to complete this assignment. You should complete Part 1 in the first lab session – build the hardware for the initial system, make small changes to the software, and run your software on the hardware that you have built.

You then have four lab sessions for Part 2, where you extend the hardware to interface with an accelerometer on the Nexys-4/A7 board, and design software to get information from the accelerometer and display it to the user. There are options here to suit teams with different hardware design skills, and to suit teams of two or three students.

A team of two students should be able to meet the minimum requirement (defined in Part 2) in the time available. A team of three students must include at least one person with experience of digital hardware design. The larger team will be expected to design a SPI master device in hardware, and to add one extra feature. In either case, you are required to work as a team, with each person taking responsibility for one part of the solution.

Tools

To work on the software for this assignment, you will use Kiel µVision development system – the version that supports ARM processors. This is installed on the lab computers as µVision 4.

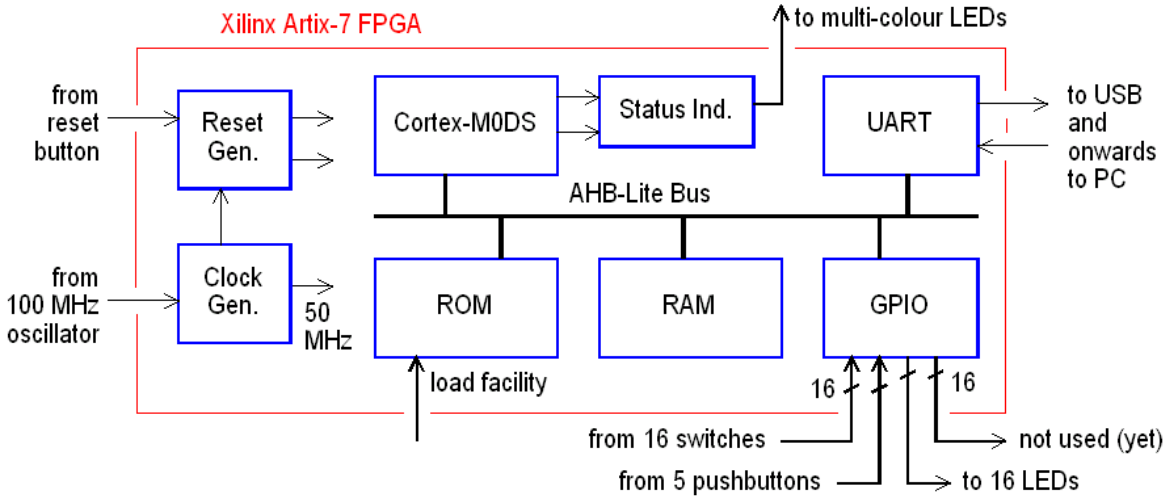
To work on the hardware, you will use the Xilinx Vivado system, version 2015.2. This is also installed on the lab computers. If you are designing hardware, and you want to run simulations outside the lab, you can use EDA Playground, which is an online system.

To download software to the program memory of your system, and to interact with that software when it is running, you will use the RealTerm software on the lab computers.

Part 1: Initial System

The aim here is to build the basic system shown below and test it. All the files that you need are in the `DES_SoC.zip` file on Brightspace. On the lab computer, extract all the files to a folder in `Documents\EmbeddedSystems\???day`. Keep the folder structure when you extract these files – it includes an entire Vivado project, a µVision project and some data sheets.

All the blocks in this system are provided – these are listed below, with brief descriptions. However, the UART block and the GPIO block are not instantiated in the top module yet. The program memory (ROM) contains a simple demonstration program at the start, but it allows you to download new software without having to rebuild the hardware.



AHBliteTop.v
This is the top-level module, where all the individual modules in the system should be instantiated and connected to the AHB-Lite bus. It is incomplete – the program memory (ROM) and data memory (RAM) are included, but the GPIO and UART peripherals are missing. See the comments, especially comments starting with `// ##`. A simple testbench is provided, which provides the clock and reset signals needed to simulate the system.

clock_gen.v
Clock generator. This provides a 50 MHz clock for all the hardware and the AHB-Lite bus, using the 100 MHz clock input from the oscillator on the Nexys-4/A7 board.

reset_gen.v
Reset generator. This provides an active-high reset signal for some of the hardware blocks in the design. This reset signal is active if the CPU RESET button is pressed. It is also active when the FPGA is first configured, and remains active until the clock generator is running. This block also provides a separate, active-low, reset signal for the processor and AHB-Lite bus, which meets the Cortex-M0 reset requirements. This reset signal is active when the hardware reset is active, when a program is being loaded into ROM, or when requested by software.

status_ind.v
Status indicator – uses two multi-colour LEDS on the Nexys-4/A7 board to display status signals.

Colour	**LD17 – on the left**	**LD16 – on the right**
Blue	Processor and AHB bus reset active	Processor sleeping (wait for interrupt)
Red	Processor lockup	ROM loader active (see below)

CORTEXM0DS.v
This is a wrapper for the Cortex-M0 DesignStart processor, provided by ARM. It instantiates the processor module, which is deliberately obfuscated. This wrapper module makes it possible to see the CPU registers in simulation.

AHBDCD.v
This is the AHB-Lite address decoder, which defines the address map. It selects the appropriate slave on the AHB-Lite bus in response to the address input. It also controls the multiplexer block, below. You will have to modify the decoder block as you add more slaves to the system.

AHBMUX.v
Multiplexers for signals from the slaves, heading back to the processor. You should not have to modify this block, but you will have to connect more slave signals to it.

AHBrom.v
Program memory, 32 kByte, read-only from the AHB-Lite bus. The memory is constructed using block RAM components on the FPGA. This memory contains the machine code for a simple test program. You can change the program later, as this program memory includes hardware (ROM loader) to allow new program code to be loaded through the serial port after the FPGA has been configured. See instructions later.

AHBram.v
Data memory, 16 kByte, with read and write access from the AHB-Lite bus. It supports byte, half-word and word (8, 16 and 32-bit) writes. This memory also uses block RAM on the FPGA.

AHBgpio.v
General-purpose input-output block. This provides two 16-bit input ports and two 16-bit output ports, with byte writes possible to the output ports. Full details, including an RTL diagram, are in a separate document, in the `Documents` folder (and also on Brightspace). A testbench for this block is provided, which simulates some AHB transactions.

AHBuart.v
UART – asynchronous serial interface block. This block can transmit and receive at 19200 bit/s, in groups of 8 data bits with one start bit, no parity and one stop bit. The transmit and receive paths each have 16-byte buffers or queues. A status register provides bits to indicate if these buffers are full or empty. Each of these status bits can be enabled to cause an interrupt, by setting appropriate bits in the UART control register. Full details are in a separate document, in the `Documents` folder (and also on Brightspace). A testbench is provided.

Constraints file
Along with all the hardware blocks, a constraint file is provided: `Nexys4(A7)_SoC.xdc`. This defines the pin numbers and signal voltages for all the connections to the FPGA. The signals not needed in this assignment are commented out.

System Integration
The system that you have been given is not complete – some of the slaves are missing. You need to build the complete system, as shown in the diagram on page 2.

Run the Vivado software. Open the project called `DemoSystem`, which is in the `Hardware` folder. All the design files are already included in the project, and you should be able to see the hierarchy of modules in the Project Manager. If you have not used Vivado before, view the introductory video on Brightspace.

If you click on Open Elaborated Design, you should see the block diagram of the system so far. You will see some warnings about unconnected ports, and you will also see these on the block diagram – those ports are for the blocks that you have yet to add.

Add the GPIO block
Instantiate the AHBgpio module in the top-level module. Connect its bus ports to the bus signals. Most of these are shared signals, and they already exist in the top module. The slave select, read data and ready output signals are unique to this block. You should create wires for those three signals, to connect the relevant ports on the GPIO block to the address decoder and to the multiplexer. You can choose any unused slave number for the GPIO block.

You will also have to connect the other ports on the block. The test program expects the `gpio_out0` port to connect to the LEDs on the Nexys-4/A7 board. The ROM loader also uses the LEDs when it is active, so a multiplexer allows the two blocks to share the LEDs. Connect the `gpio_out0` port to the multiplexer input, using existing wires called `led_gpio`.

Connect the signals from the switches on the Nexys4 board, `sw`, to port `gpio_in0`. Connect the signals from the push-buttons to the 5 least significant bits of `gpio_in1`, with 0s on the other bits. There is already a signal called `buttons` that combines the 5 button signals.

Add the UART block
Instantiate the AHBuart module in the top-level module. Connect its bus ports to bus signals, including some new wires, as for the GPIO block. You can use any unused slave number.

Connect the serial transmit and receive ports to ports of the top-level module, called `serialTx` and `serialRx`. On the Nexys-4/A7 board, these signals connect to a virtual serial port, which is part of the USB interface. The software uses this port to communicate with the PC.

Connect the UART interrupt request output to the processor. The Cortex-M0 has a 16-bit interrupt request signal called `IRQ`, so it can accept interrupts from up to 16 sources. The software expects the UART interrupt at `IRQ` bit 1. As there are no other interrupt sources (yet), you should connect all the other bits of the `IRQ` signal to 0.

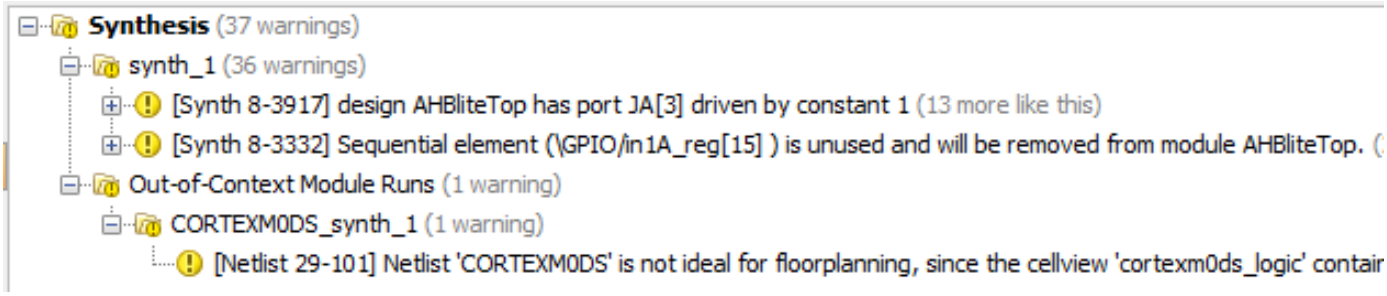
Address decoding
Now modify the address decoder block to put the new slaves at the correct places in the address map. The address decoder is designed to check only the 8 most significant bits of the address, so each slave gets a 16 MB range of addresses – far more than it needs.

You should see that the program memory (ROM) is already located at address `0x00000000`, and the data memory (RAM) at `0x20000000`. You need to add logic to put the GPIO block at `0x50000000` and the UART block at `0x51000000`, to work with the demonstration software.

Check the elaborated design again. Note that you can expand any block to see what is inside it.

Implementing Your System on FPGA
In Vivado, Synthesis Settings, choose Vivado Synthesis Defaults. Then Run Synthesis. The first run may take a long time, as all the blocks may have to be synthesised. The project is configured so that the processor, ROM and RAM will not need to be synthesised again. You can set synthesis to run in the background if you want to get on with other tasks.

When the synthesis has run, view the Messages tab at the bottom of the Vivado window. Tick the boxes to show Errors and Warnings, but not information messages (there are too many of those). You need to check that all the warnings are acceptable before you proceed.



The warnings above are normal at this stage: There are some output ports that are not used yet, and those ports are driven by constant values. GPIO input 1 has 16 bits, but only 5 bits are used at present. Each input bit has a synchroniser circuit with two flip-flops, so there are 22 redundant flip-flops. There are also registers in the bus interface in each slave, storing the same signals, so some of those can be removed. This version of the Cortex-M0 Verilog is deliberately obfuscated by ARM, to protect the design.

If there are errors, or more warnings, first click on the bin symbol beside the messages tab – that will remove any old messages. Then fix any problems that remain. When you are happy with the synthesis results, Run Implementation. This may take some time.

When Implementation completes, with no warnings, the next step is Generate Bitstream. This produces the `.bit` file that you need to download to the FPGA to configure it with your design.

Configure the FPGA
Use the Hardware Manager in Vivado. Connect the Nexys-4/A7 board to your computer using the USB cable. Switch on – the power switch is near the top left corner of the board.

In the Hardware Manager, select Open Target. The first time you do this, you should choose Open New Target. Accept the defaults, and the Nexys4 board should be found and identified. The FPGA on the board should also be found. When you do this again, when you click Open Target, you should see the FPGA board listed as a Recent Target – just select it.

When you have opened the target board, click Program Device, and select the only device offered. The correct bitstream file will usually be offered. This file should have the name of your top-level module, with extension `.bit`. You do not need a Debug Probes File now. If all is correct, click Program.

Your design should now be on the FPGA, and ready for testing. If you switch off the board, the configuration will be lost, and you will have to download the `.bit` file again: Program Device. There is no need to repeat synthesis and implementation if the design has not changed.

Testing
When the FPGA has been configured, all of your hardware should be on the chip. Everything will have been reset, and the example software in the program memory should be running on the processor. Some of the LEDs might be on – they should reflect the state of the switches.

You can change the switches, then press the `CPU RESET` button to restart the program. The LEDs should flash and then match the new state of the switches.

The right-most status LED (LD16) should be showing blue – this indicates that the processor is sleeping, waiting for an interrupt. The interrupt would be caused by a character being received on the serial interface. For other status indications, see the table on page 2.

RealTerm
Run the RealTerm software. On the Display tab, set the display mode to Ansi. Increase the number of Rows to 30 and tick the box to allow Scrollback.

On the Port tab, choose the port number corresponding to the virtual serial port on the Nexys-4/A7 board. This may be different on each computer – it should be the only number between 4 and 9. It will usually be followed by `\VCP0`

Configure the port for 19200 bit/s (shown as Baud), 8 data bits, no parity, 1 stop bit, no handshake. Click the Change button so that these changes take effect.

Now press the `CPU RESET` button again. You should see a message appear in the RealTerm window. Click the mouse in the black part of the RealTerm window and type some characters. You should see those characters echoed back and displayed in RealTerm. When you press return, all your input will be printed, with the letters modified.

The rightmost LEDs on the Nexys-4/A7 board should show each character code as it is received. When you press return, all the LEDs will flash a pattern that depends on the switch settings.

If all of this is working, your initial hardware design is correct, and you are ready to move to the software side.

Software
Run the uVision software. Open the `DES_M0_SoC` project located in the `Software` folder. Check the project options – on the Target tab, you should see the ROM and RAM areas defined. Check the path to the System-Viewer `.sfr` file – it should point to a file in your `Software` folder.

The project includes four demonstration programs, with `main.c` as the active file. Examine `main.c` – this is the program that was already loaded into the program memory in your system, and is now running on the processor in the FPGA on the Nexys-4/A7 board. The definitions of the hardware registers are in the header file `DES_M0_SoC.h`. You should be able to understand why the system is behaving as it is.

Note the "critical section", where interrupts are disabled to prevent any change to the `RxBuf` array while it is being processed. Interrupts should only be disabled for a short time, so the printing of the `TxBuf` array, which will take a long time, is done outside the critical section.

Modify the program
When you understand `main.c`, change it. Make it send a different welcome message, so you will recognise the new version. Modify it so that it reports the number of characters that you have typed, and also reports the state of the switches as a number, in decimal or hexadecimal.

Build your new program. You should find an output file called `ROMcode.txt` in the Software folder. Examine this – it contains the 32-bit words that should be in the program memory.

The first word defines the initial value of the stack pointer. The second is the address of the first instruction to be executed after reset. The 18th word should be the address of the interrupt handler for the UART interrupt. (This will be explained in lectures soon.)

Download to program memory
On the Nexys-4/A7 board, press the black pushbutton marked `BTNU`. While holding this down, press and release the reset button. Then release `BTNU`. This activates the ROM loader hardware, which keeps the processor in reset. The status indicators should show blue and red.

In RealTerm, on the Send tab, under Dump File to Port, select the `ROMcode.txt` file. Click the Send File button to start the download. The entire content of the file will be transmitted, byte by byte. The ROM loader should receive those bytes – you should see the LEDs on the Nexys board displaying a count of the words received.

When the file has been sent, click in the RealTerm terminal window (the black part) and press Q on the keyboard. This tells the ROM loader that the transfer is complete. The status indicators should change as the processor starts running your new program.

Check that your new program behaves as expected.

Review
When you have completed the initial tasks, and before you start on the system design, submit your modified Verilog files: `AHBliteTop.v` and `AHBDCD.v` for review. Also submit your modified program file, `main.c` for review. Use Notepad++ to print these files to a pdf document (or three separate documents), so that they can be annotated in Brightspace. Then upload them as an Assignment submission in Brightspace: SoC Review. You need to join one of the SoC groups in Brightspace to access this.

Your submission will be returned with feedback, but will not be graded.

Part 2: System Design
First decide what you want the system to do – see the requirements below. Then you can start designing a system to do it.

First break the problem into a few large blocks, and think about the information flows between the blocks. Decide what will be done by hardware and what will be done by software. Then design one block at a time – if necessary, divide a block into smaller blocks…

Minimum Requirement

There is a 3-axis accelerometer on the Nexys-4/A7 board, an Analog Devices ADXL362, which has a SPI interface. The minimum requirement for this assignment is to configure and use the accelerometer to measure acceleration on one axis, and to display this acceleration on a line of 16 LEDs on the Nexys board. Already you can break this into two separate problems: how to measure acceleration and how to display the information on a line of LEDs.

Display

The display is usually the easier problem. Section 9 of the Nexys-4/A7 data sheet describes the LEDs on the board. The line of 16 LEDs is already connected to a GPIO output port, so you can control the LEDs in software.

You need to map a number to a pattern of LEDs turned on and off. For example, you could use a one-hot code, with exactly one LED on at a time, in a position that varies with acceleration. Or you could use a thermometer code, with a set of adjacent LEDs on, where the number of LEDs turned on varies with acceleration.

To complete the design, you need to know the meaning of the number that you are going to display. Is it signed or unsigned? What units is it in? What range of values do you expect it to have? Then you can scale it to fit the type of display that you have chosen.

Acceleration

Measuring acceleration means using the accelerometer. The data sheet for the Analog Devices ADXL362 accelerometer is in the `Documents` folder, and also on Brightspace. Someone in the team should look at this data sheet. There is no need to read every word at this stage – just get an overview of what information is available from the accelerometer, how to configure the accelerometer to provide the information that you want, and how to access that information.

This part of the design breaks into at least two distinct problems, which you can solve separately:

- The low-level problem is how to communication with the accelerometer – sending and receiving bytes using the SPI interface, without thinking about what those bytes mean.
- The higher-level problem is deciding what information you want to send to the accelerometer and collect from the accelerometer, assuming that communication is possible.

Simple software to simulate the accelerometer is provided on Brightspace, so that you can test your higher-level software without access to the hardware. It provides two functions, `SPIselect()` to control the slave select signal and `SPIbyte()` to transfer a byte. If you want to use the simulator, you should design your software using functions with the same names and the same effects.

Accelerometer Details

The accelerometer is mounted on the underside of the Nexys circuit board, marked as IC13. It is oriented so that the Z axis is perpendicular to the circuit board, and the Y axis is parallel to the line of 16 LEDs.

In SI units the letter g means gram. In the world of acceleration measurement, the letter *g* is used to denote acceleration due to gravity at the surface of the earth, approximately 9.81 $m \cdot s^{-2}$. You will also see m*g* being used for one thousandth of that acceleration.

You can configure the accelerometer to measure acceleration in one of three ranges, ±2 *g*, ±4 *g*, ±8 *g*. Those range names are approximate – the exact mapping from acceleration to accelerometer output number is given in Table 1, on page 4 of the data sheet. The Sensitivity section gives the acceleration that corresponds to one LSB (a change of 1 in the output number), and the Scale Factor, which is the reciprocal of this – the change in output number that corresponds to a change in acceleration of 1 *g*.

Extending the System

These are suggestions for extending the basic system. You may choose to extend the system in other ways if you prefer. A team of three will be expected to add one extension.

- You could use the 8-digit display to show acceleration in numerical form. Ideally, this should be in meaningful units, such as *g* or $m \cdot s^{-2}$ (the SI standard). Four digits would give adequate resolution, even with a sign symbol, so you could use the eight digits to display acceleration in two axes if you wished.

- You could use some switches to allow the user to configure the system, perhaps selecting which axis of acceleration to display, or which units to use.

- You could use the serial interface to provide a user interface on your computer, using RealTerm to display information from the accelerometer, and perhaps to allow the user to input commands on the keyboard to configure the system. You have an example program with user input: `main-scanf.c`

- You could use the accelerometer to detect acceleration above some threshold – this would be common in collision detection in vehicles, knock detection in intruder alarms, or step counting in fitness trackers. (The accelerometer has features to allow activity detection.) You could flash an LED each time a significant acceleration was detected. You could display a count of the number of times this occurred.

Detailed Design

SPI Communication

The accelerometer has a SPI interface – it will act as a slave. To communicate with it, you need something that will act as a SPI master. You could connect the SPI signals to GPIO pins in your system, and then implement the SPI master protocol in software. The software functions to do that could form one block in your design.

A better solution would be to design a hardware SPI master to connect to the AHB-Lite bus – this could handle all the SPI signals, with the correct timing, and provide some registers that the software could access to control the interface and to transfer bytes. A team of three will be expected to design a hardware SPI master.

It would be normal to design the hardware to transfer bytes to and from a slave device. The sequences of bytes used for a particular slave device should be handled in software. That keeps the hardware simple and allows it to be used with many slave devices.

If the hardware handles the low-level parts of the SPI protocol, the software functions to send and receive bytes will be much simpler. The higher-level functions in your software will be the same, regardless of how you solve the low-level part of the communication problem.

For this part of the design, you need to know how the accelerometer is connected to the FPGA. This is described in section 13 of the Nexys-4/A7 data sheet, which is in the `Documents` folder, and also on Brightspace. The four SPI signals to and from the accelerometer are already connected to ports on the top module in your initial hardware design.

Interrupts

Another design decision will be on interrupts. What interrupts will you use in your system?

You have an example using an interrupt to alert the processor when data has arrived on the UART interface – this is useful because this data arrives at times that are outside your control, when someone presses a key on the keyboard of the computer.

If you are designing a hardware SPI interface, you could use another interrupt to indicate that data had arrived on the SPI interface. However, this would not be particularly useful, as this data only arrives when you request it, so you already know that it is coming.

You could arrange for the accelerometer to cause an interrupt when it has detected some special event. This could be useful in one of the extensions above. You would have to connect one of the interrupt output pins on the accelerometer to a new input port on your top module…

If you want an interrupt at regular intervals, to make your software do something at a known rate, you can use the SysTick timer, which is part of the processor, as described in lectures on the Cortex-M0. It can be configured to cause interrupts at rates from about 3 Hz upwards.

The original example program works directly with registers of the interrupt controller (NVIC). You can also work directly with registers for the SysTick timer – see `main-SysTick.c`. Alternatively, you can use the CMSIS structs and functions to configure the NVIC and SysTick. (CMSIS functions are provided by ARM, to work with the processor hardware, and will be covered in lectures soon.) See the examples in `main-CMSIS.c`

8-digit Display

The 8-digit 7-segment display is described in section 9 of the Nexys-4/A7 data sheet. The display signals are multiplexed: there are 8 segment signals to control which pattern is displayed at any point in time, and 8 digit signals to control where that pattern is displayed.

If your design includes this display, you need to decide how to interface to it. You could connect those 16 signals to a 16-bit GPIO port, and control the display in software. Or you could connect those signals to a hardware display block, and connect that to the AHB-Lite bus in your system. A suitable block is provided (`AHBdisp.v`), along with a testbench and basic documentation.

The display signals are NOT connected to your initial top module. If you want to use the display, you need to add two ports to the top module, and also uncomment the relevant signals in the constraint file, which maps the signal names to pins on the FPGA.

Teamwork

When you have completed the top-level design, divide the block design tasks among the members of the team. For example, someone needs to do the detailed design of the SPI master interface, either as hardware or as a set of software functions, operating at byte level. That person will need to read about the SPI interface in the accelerometer, and consider bit rate, clock polarity, clock phase, etc.

Someone else might write the software that communicates with the accelerometer at a higher level. That person will read different parts of the accelerometer data sheet, dealing with registers in the accelerometer, how to configure the accelerometer, and how to get the information that you want from the accelerometer.

Someone needs to extend the top-level hardware description to suit your overall design. At a minimum, this will involve connecting the accelerometer signals to some GPIO signals. It may involve adding more hardware blocks and connecting display signals, with corresponding changes to the ports of the top module and to the constraint file...

Someone needs to write software to implement the display on the line of LEDs. If there is a user interface, someone needs to design that, and implement it in software.

You should make a plan that will allow the team to complete the rest of the work in the time available – that will be four lab sessions if you have completed Part 1 on time.

Report

You will work on this assignment as part of a team, but you must submit your own report. Block diagrams, RTL diagrams and flow charts may be shared within the team, with credit given to whoever created the diagram. The text of the report should be your own.

The report is due by 23:59 on 1st May. Standard penalties apply for late submission. Submit your report as a pdf document, as the SoC Report assignment in Brightspace.

Your report should start with the usual declaration of authorship – a template will be provided on Brightspace. Your report should include:

- An overview of your design, with details of the information that you can display, the user interface, etc. Also details of the design process: the main blocks that you identified, your decisions on how each block would be implemented, etc. This section should make clear who worked on each block.
- Details of the blocks that you designed. For software blocks, explain how you broke the problem into separate functions, and what each function does. Use a flow chart or diagram if you wish. For hardware blocks, give an RTL diagram, and explain how the block works and why you designed it that way.
- A brief description of the tests you performed to verify that your system works as intended. For hardware blocks that you designed, include an example timing diagram from simulation.

The Verilog and program code will be collected separately, but you may want to include some code in your report, to show how you solved a particular problem. In that case, format the code in a fixed-pitch font, so that indentation is preserved. Use a small font, and shorten long lines, so that lines do not wrap. There is no need to leave a blank line between every two lines of code, but a blank line can be useful to separate different sections of code. The template document has a style called Code that you can use for this.

One member of the team should submit all the source files used in the assignment: Verilog design files, testbench files and program files. Submit these as the SoC Code assignment in Brightspace. You need to join one of the SoC groups in Brightspace to access this.

The Code assignment will accept any number of files, but if the number is large, you may find it more convenient to combine all the files into one zip file and submit that. Do **not** include the entire project folder – that will be many megabytes in size, and only a few kilobytes are needed.

Grading

Grading will take into account:

- The capability of your system (what can it do?), relative to the size of your team.
- The design decisions that you made – is this a sensible way to solve the problem?
- The quality of the implementation: well-structured software will be expected, along with well-designed hardware where relevant.
- The quality of the documentation: both your report and the comments on the code.